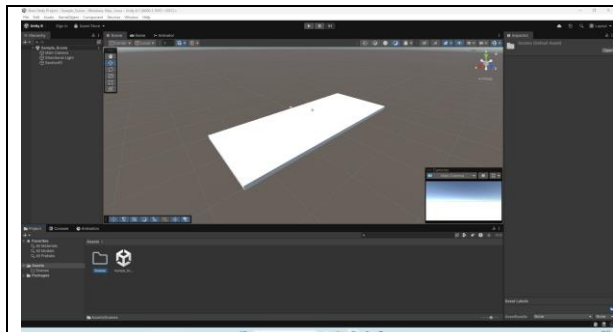
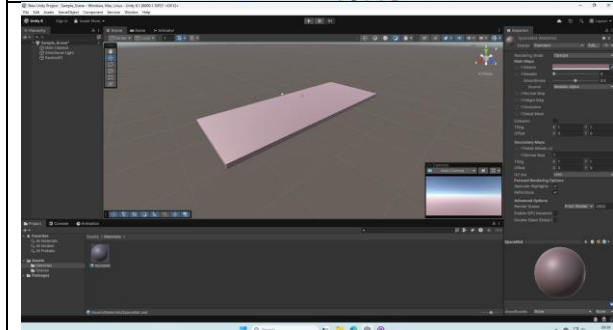




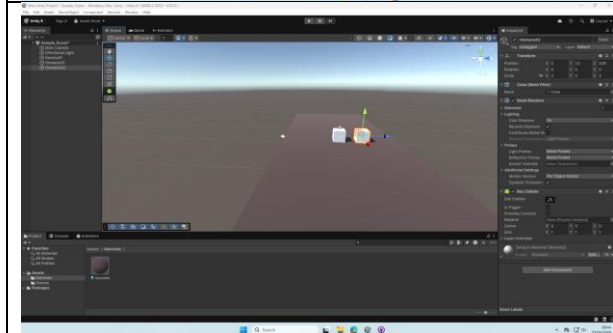
The Development process has now begun and i have created a basic platform to use as a testing ground and a starting point.



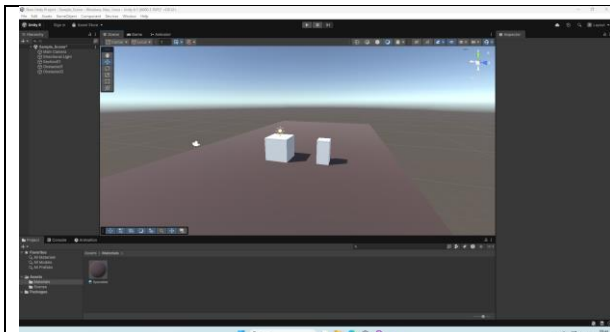
I added a texture to this platform to add some color and maybe some textures in the future



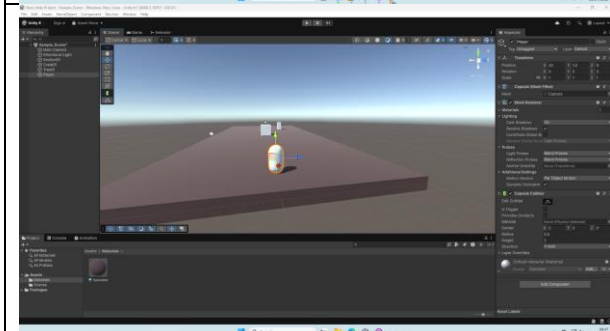
I added a box which will be my player in the future



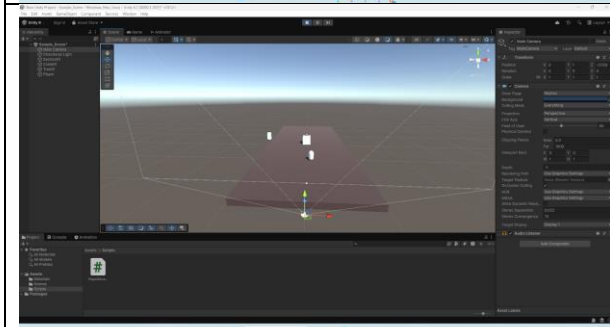
Im creating sme obstacles here with the corret names and components



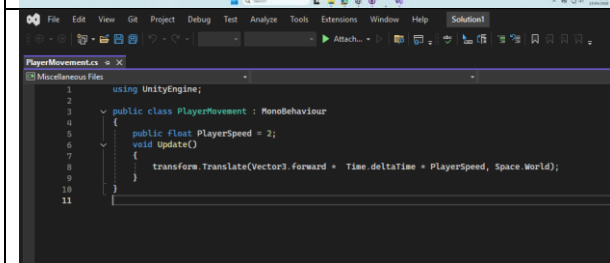
I reshaped them to simulate them better in the game



I created a capsule for the player as its the better object to use



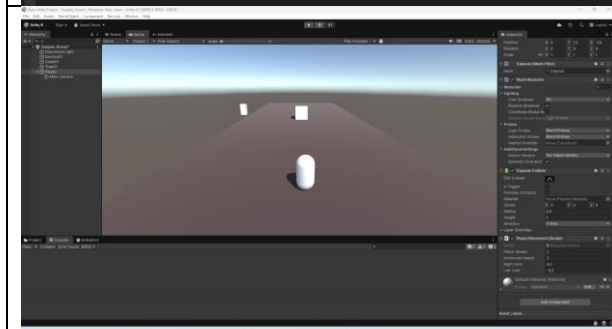
I moved the camera so that it is behind the player and shows the level ahead



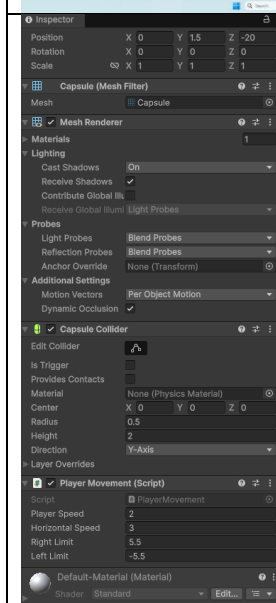
I created a basic player oemen script to allow the player to have some basi c movement

```
1 using UnityEngine;
2
3 public class PlayerMovement : MonoBehaviour
4 {
5     public float PlayerSpeed = 2;
6     public float horizontalSpeed = 3;
7     public float rightLimit = 5.5f;
8     public float leftLimit = -5.5f;
9
10 void Update()
11 {
12     transform.Translate(Vector3.forward * Time.deltaTime * PlayerSpeed, Space.World);
13     if (Input.GetKey(KeyCode.A) || Input.GetKey(KeyCode.LeftArrow))
14     {
15         if (this.gameObject.transform.position.x > leftLimit)
16         {
17             transform.Translate(Vector3.left * Time.deltaTime * horizontalSpeed);
18         }
19     }
20     if (Input.GetKey(KeyCode.D) || Input.GetKey(KeyCode.RightArrow))
21     {
22         if (this.gameObject.transform.position.x < rightLimit)
23         {
24             transform.Translate(Vector3.right * Time.deltaTime * horizontalSpeed);
25         }
26     }
27 }
28 }
```

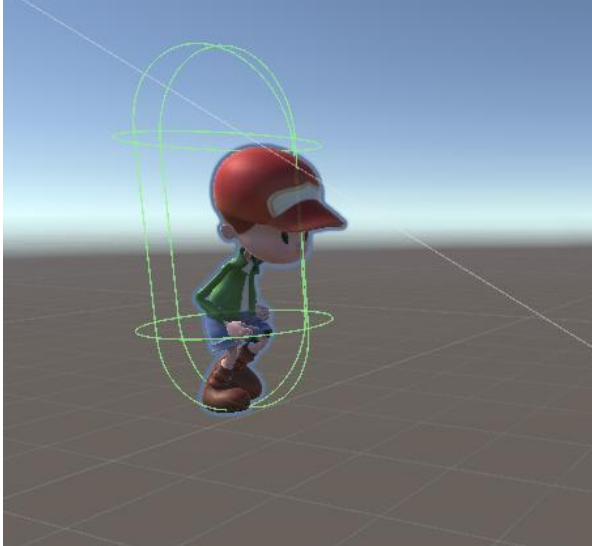
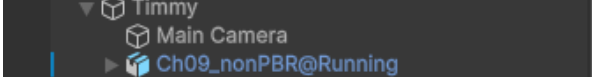
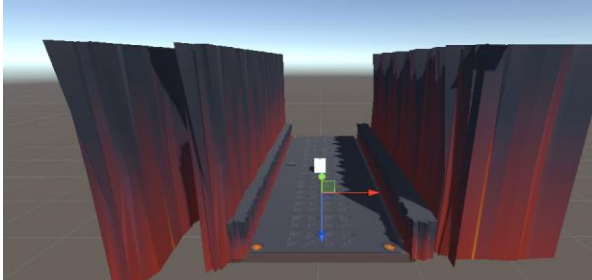
Im now making limits on how far the player can move left and right to ensure they don't go off the platform

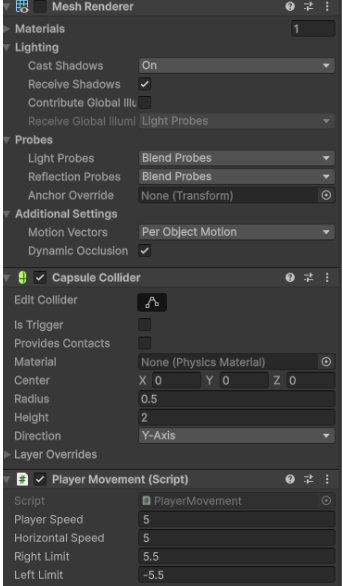
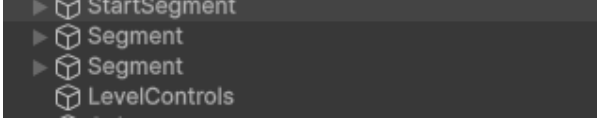


I added the script to the player and tested it to ensure the variables were correct for the dimensions of the game



I had to edit some components on the player to ensure it all worked because it was moving in weird angles so I had to freeze certain rotations and directions

	I added a sprite to the player capsule and adjusted the collider to match the size of the sprite and transferred the components over to it
	As you can see here
	I began level building as i was getting tired of constantly scripting and coding as i enjoy designing the most in games development
	I created a basic coin just to get some ui into my game
<pre>using UnityEngine; public class CollectibleRotate : MonoBehaviour { [SerializeField] int rotateSpeed = 1; void Update() { transform.Rotate(0, rotateSpeed, 0, Space.World); } }</pre>	Here is the script which i added to the coin that allowed it to spin in a cool smooth animation

<pre>using UnityEngine; public class PlayerMovement : MonoBehaviour { public float PlayerSpeed = 2; public float horizontalSpeed = 3; public float rightLimit = 5.5f; public float leftLimit = -5.5f; void Update() { transform.Translate(Vector3.forward * Time.deltaTime * PlayerSpeed, Space.World); if(Input.GetKey(KeyCode.A) Input.GetKey(KeyCode.LeftArrow)) { if (this.gameObject.transform.position.x > leftLimit) { transform.Translate(Vector3.left * Time.deltaTime * horizontalSpeed); } } if(Input.GetKey(KeyCode.D) Input.GetKey(KeyCode.RightArrow)) { if (this.gameObject.transform.position.x < rightLimit) { transform.Translate(Vector3.right * Time.deltaTime * horizontalSpeed * -1); } } } }</pre>	I had to add additions to the player movement script here
 The screenshot shows the Unity Inspector for a player object. The Mesh Renderer component is at the top. Below it is the Capsule Collider component, which is configured with a radius of 0.5 and a height of 2, with the direction set to Y-Axis. At the bottom is the Player Movement (Script) component, which has four public variables: Player Speed (5), Horizontal Speed (5), Right Limit (5.5), and Left Limit (-5.5).	I also edited the components to allow e to detect and pick up coins
<pre>using System.Collections; using System.Collections.Generic; using UnityEngine; public class SegementGenerator : MonoBehaviour { public GameObject [] segment; [SerializeField] int zPos = 50; [SerializeField] bool creatingSegment = false; [SerializeField] int segmentnum; void Update() { if (creatingSegment == false) { creatingSegment = true; StartCoroutine(SegmentGen()); } } IEnumerator SegmentGen() { segmentnum = Random.Range(0, 3); Instantiate(segment[segmentnum], new Vector3(0, 0, zPos), Quaternion.identity); zPos += 50; yield return new WaitForSeconds(3); creatingSegment = false; } }</pre>	I now needed to allow the segm,ents in my game to generate the ground infinitely to simulate an endless runner properly
 The screenshot shows the Unity Hierarchy panel. It contains a list of objects: StartSegment, Segment, Segment, and LevelControls. Each object is represented by a cube icon.	I had to clone and edit the components and then edit them to i could de frenciate them

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

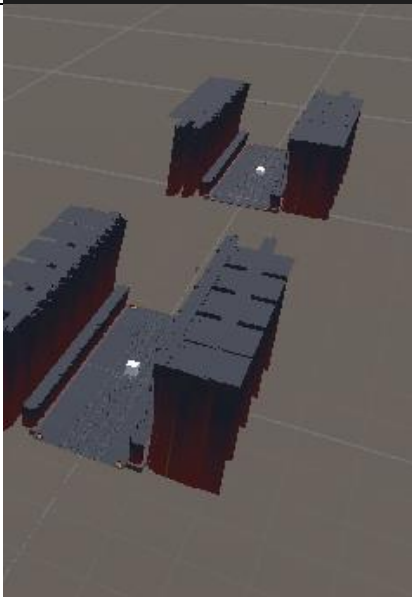
public class SegementGenerator : MonoBehaviour
{
    public GameObject [] segment;
    [SerializeField] int zPos = 50;
    [SerializeField] bool creatingSegment = false;
    [SerializeField] int segmentnum;

    void Update()
    {
        if (creatingSegment == false)
        {
            creatingSegment = true;
            StartCoroutine(SegmentGen());
        }
    }

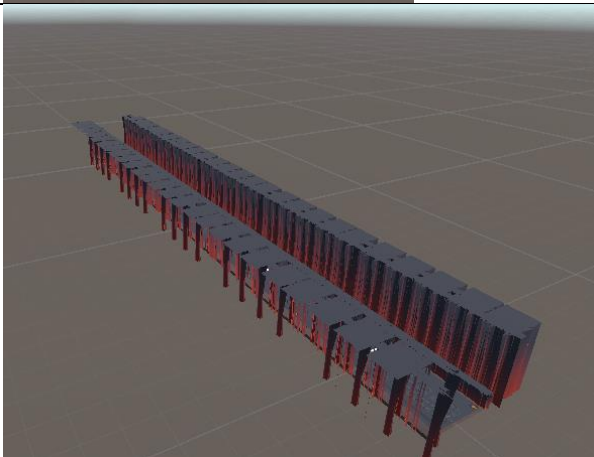
    IEnumerator SegmentGen()
    {
        segmentnum = Random.Range(0, 3);
        Instantiate(segment[segmentnum], new Vector3(0, 0, zPos), Quaternion.identity);
        zPos += 50;
        yield return new WaitForSeconds(3);
        creatingSegment = false;
    }
}

```

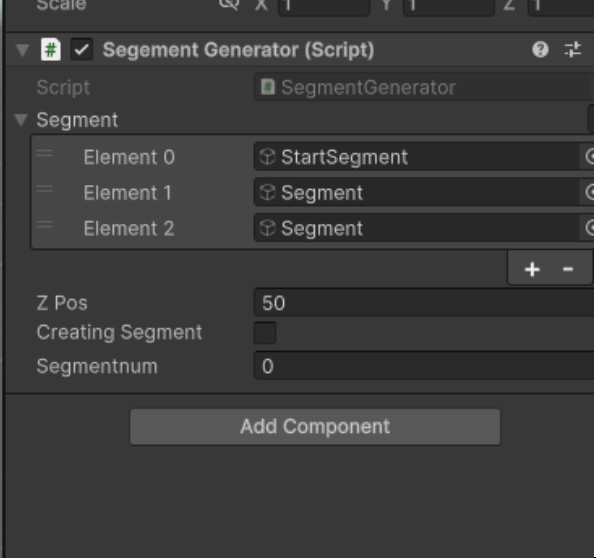
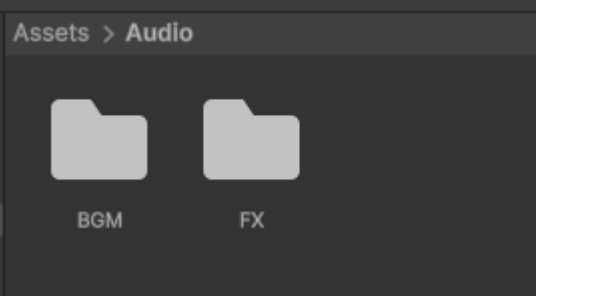
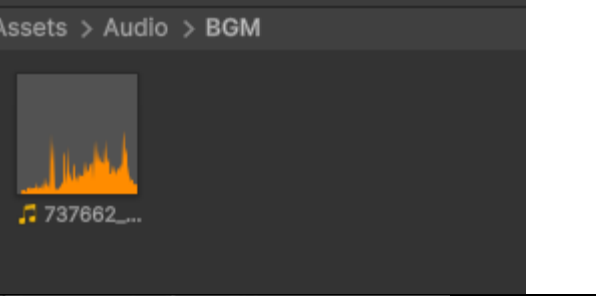
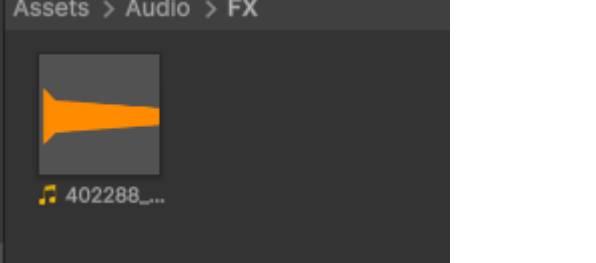
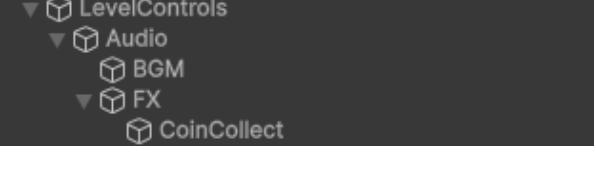
After a few errors i had to edit the script to ensure it would run, for some odd reason it was spawning in random locations and causing unity to lag

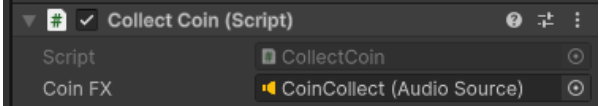
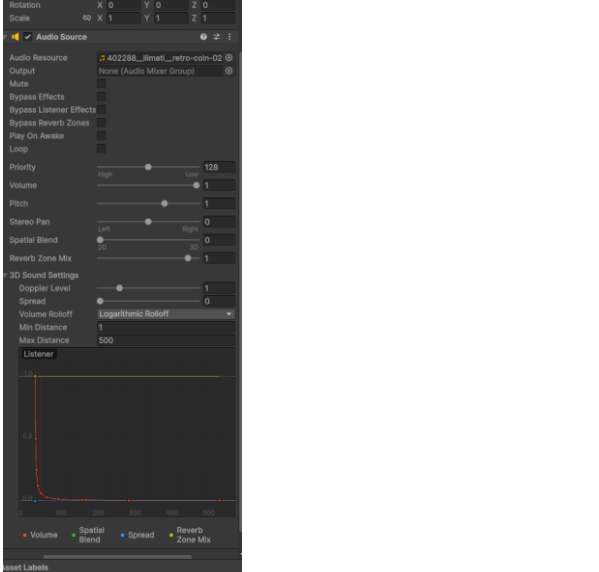
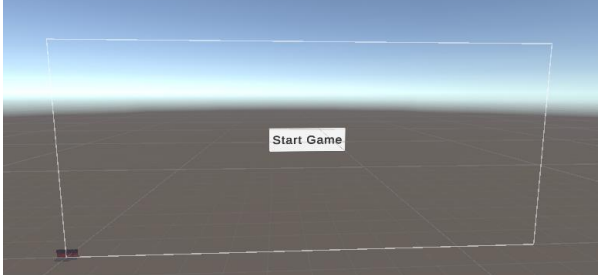
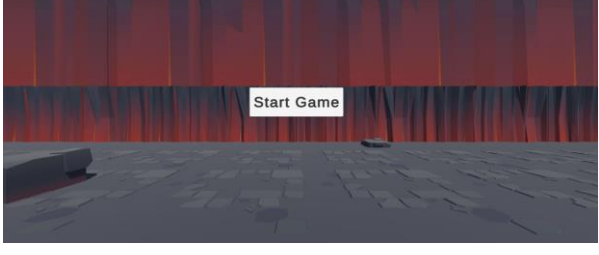
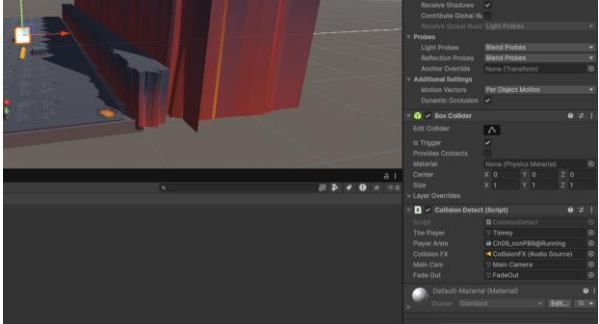


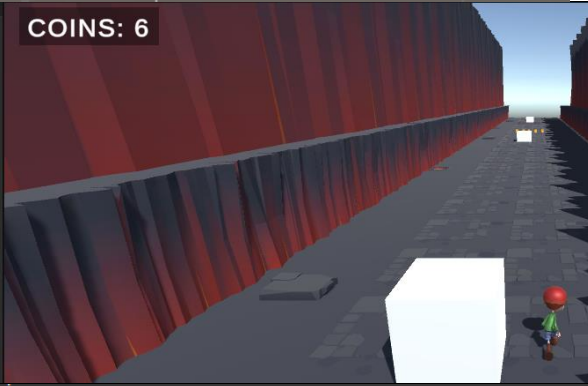
Here are the segments

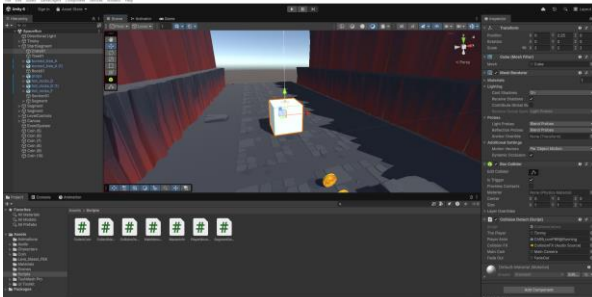
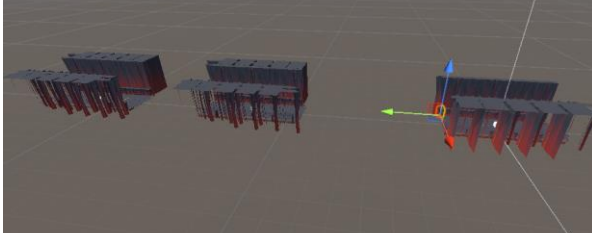
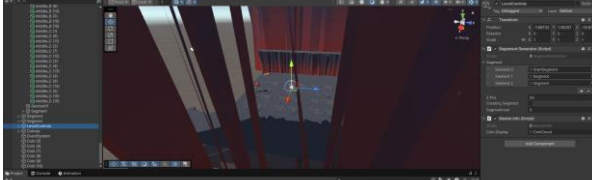
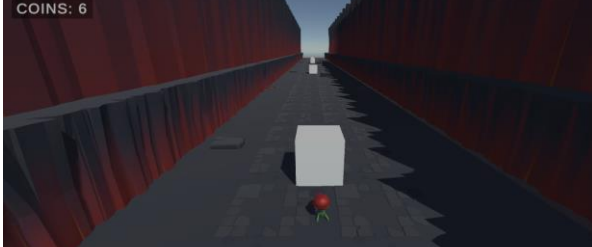


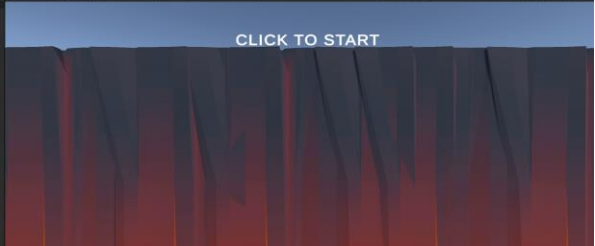
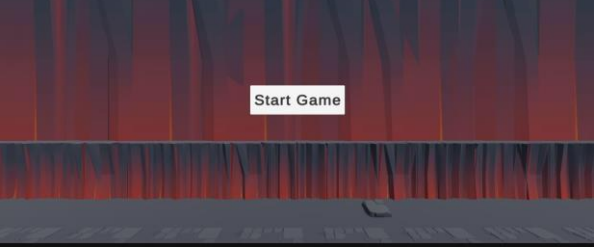
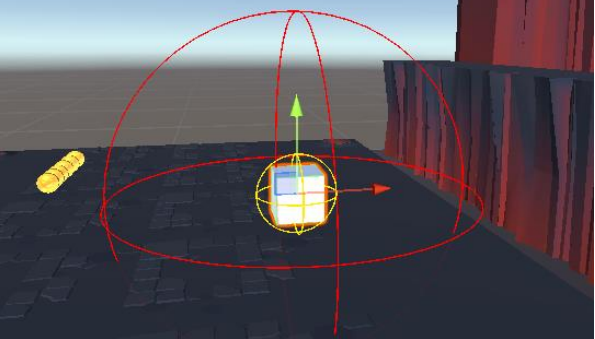
Here is the code wroking alongside the segments generating them in a straught line without any errors

	I had to add the segments i was using to the componet script to ensur ethat they would all work together
	Here are my orgajsed files for background music and sound affects
	Here are some of the audios
	
	Here is the new method i learnt of how to do these audios by using the hierarchy to get tghem domne

	Here is the script which lets the audio play
	Here are my audio soucrs
	Im now working on the main menu as it is a crucial thing to add to a game i have created a basic canvas and added a button saying start meny
	I created a cool amimation where the camera will drop and u can see the backgriund of the game and then u need to click start game jiust a cool trick i learnt
	I am now working on my obstacles in the game by adding in the correct components to the script to ensure it collides correctly

	Here is the UI for the coins collection I have edited it for a better user experience
	Here you can see it going up in game with the correct amount being displayed
<pre> using System.Collections; using System.Collections.Generic; using UnityEngine; [Unity Script (11 asset references) 0 references] public class CollectCoin : MonoBehaviour { [SerializeField] AudioSource coinFX; [Unity Message 0 references] void OnTriggerEnter(Collider other) { coinFX.Play(); MasterInfo.coinCount += 1; this.gameObject.SetActive(false); } } </pre>	Here is my coin collect script which portrays the amount in the UI and updates it
<pre> using UnityEngine; [Unity Script (1 asset reference) 1 reference] public class MasterInfo : MonoBehaviour { public static int coinCount = 0; [SerializeField] GameObject coinDisplay; [Unity Message 0 references] void Update() { coinDisplay.GetComponent<TMP_Text>().text = "COINS: " + coinCount; } } </pre>	This script allows the UI to work on the Canvas

<pre> using System.Collections; using System.Collections.Generic; using UnityEngine; using UnityEngine.SceneManagement; // Unity Script (3 asset references) 0 references public class CollisionDetect : MonoBehaviour { [SerializeField] GameObject thePlayer; [SerializeField] GameObject playerAnim; [SerializeField] AudioSource collisionFX; [SerializeField] GameObject MainCam; [SerializeField] GameObject fadeOut; // Unity Message 0 references void OnTriggerEnter(Collider other) { StartCoroutine(CollisionEnd()); } // 1 reference IEnumerator CollisionEnd() { collisionFX.Play(); thePlayer.GetComponent<PlayerMovement>().enabled = false; playerAnim.GetComponent<Animator>().Play("Stumble Backwards"); MainCam.GetComponent<Animator>().Play("CollisionCam"); yield return new WaitForSeconds(3f); fadeOut.SetActive(true); yield return new WaitForSeconds(3); SceneManager.LoadScene(0); } } </pre>	Here is the collision detect script which allows the player to detect colliders and to die
	Here I have added everything to this obstacle and I play tested it and it works perfectly
	I am now working on the level design as it is the aspect of the game which is the most behind
	Here I am doing more level designing as I noticed unnecessary things in my design that make the game lag
	Here I coded it so that when the player collides with the obstacle it shakes the camera and creates a fading animation

	It then takes the player. To the start menu which is a looping game
	Whne the player clicks start it drops the camera to the start game button
	Here is the moving enemy ai which can move to the player if it is in that range
<pre> using UnityEngine; [Serializable] public class EnemyAI : MonoBehaviour { [Header("Target")] [SerializeField] private Transform target; [SerializeField] private string targetTag = "Player"; [Header("Movement")] [SerializeField] private float moveSpeed = 3f; [SerializeField] private float rotationSpeed = 18f; [SerializeField] private float stoppingDistance = 1f; [SerializeField] private float chaseRange = 20f; private Rigidbody _rb; private Vector3 _rbVelocity; [UnityMessage] void Start() { if (target == null) { var playerObj = GameObject.FindWithTag(targetTag); if (playerObj != null) { target = playerObj.transform; } } _rb = GetComponent<rigidbody>(); } [UnityMessage] void Update() { if (target == null) { return; } Vector3 toTarget = target.position - transform.position; float distance = toTarget.magnitude; // Only chase when inside ChaseRange if (distance > chaseRange) { _rbVelocity = Vector3.zero; return; } Vector3 direction = toTarget.normalized; // Smoothly rotate to face the target if (direction.sqrMagnitude > 0f) { Quaternion targetRot = Quaternion.LookRotation(direction); transform.rotation = Quaternion.Slerp(transform.rotation, targetRot, rotationSpeed * Time.deltaTime); } } </rigidbody></pre>	Here is the code for it which took a really long time to build

```

Quaternion targetRot = Quaternion.LookRotation(direction);
transform.rotation = Quaternion.Slerp(transform.rotation, targetRot, rotationSpeed * Time.deltaTime);

// Move toward the target until within stoppingDistance
if (distance > stoppingDistance)
{
    if (_rb != null)
    {
        // Defer physics movement to FixedUpdate
        _rbVelocity = direction * moveSpeed;
    }
    else
    {
        transform.position += direction * moveSpeed * Time.deltaTime;
    }
}
else
{
    _rbVelocity = Vector3.zero;
}

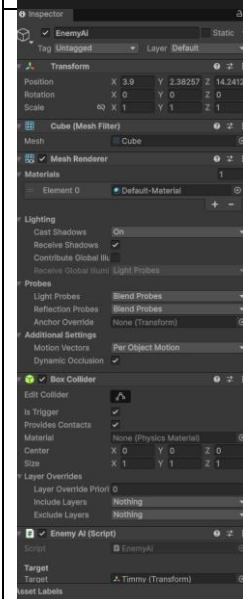
// Unity Message() references
void FixedUpdate()
{
    if (_rb == null)
    {
        return;
    }

    if (_rbVelocity.sqrMagnitude > 0f)
    {
        _rb.MovePosition(_rb.position + _rbVelocity * Time.fixedDeltaTime);
    }
}

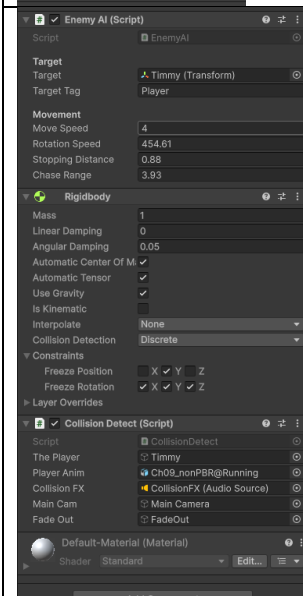
// Unity Message() references
void OnDrawGizmosSelected()
{
    Gizmos.color = Color.red;
    Gizmos.DrawWireSphere(transform.position, chaseRange);
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(transform.position, stoppingDistance);
}

```

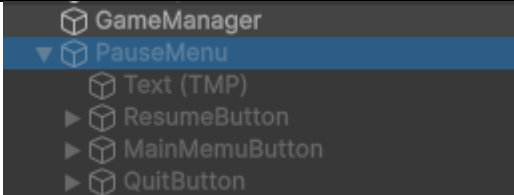
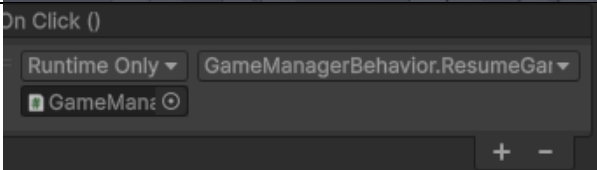
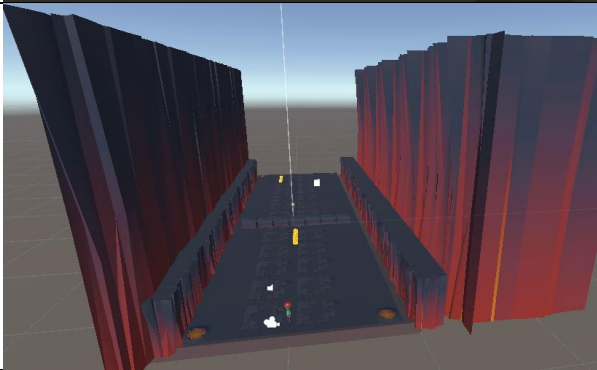
Second page



Here is the components for the enemy ai a lot and to be done but eventually it worked out well



I had to play around with the variables on the enemy ai to ensure the game isn't too hard or too easy

	Here is the pause menu components
	Here is the pause menu in action
	Here is the on click for the pause menu
	All things are set and ready for my game!